Phase 1 Relaxation (PGD) Timing Profile

Where the Γ-Convergence Solver Spends Its Time, and What That Implies for Scaling the Number of Regions

June 2026

Abstract

This document records empirical timing measurements collected by the `RelaxationProfilingState` instrumentation in `src/profiling.py` for Phase 1, the projected-gradient-descent (PGD) relaxation of the Γ-convergence energy. Across a 3×2 sweep (penalty weight $\lambda \in \{1.9, 2.0, 2.1\}$, two random seeds) on a 30-region torus, a single callback — `projection`, the orthogonal projection onto the partition and equal-area constraints — consumes 86–88% of wall time in *every* run. The total runtime varies by 4.3× (923 s to 3401 s) not because of mesh size but because of *line-search thrashing*: in the slow regime the Armijo backtracking takes ∼11 trial steps per iteration, and because the solver re-projects on every trial step, the projection count explodes. The practical conclusion for scaling up N (the number of regions) is that projection cost, not FEM assembly, is the quantity to attack: re-projection on backtrack and the per-call inner-iteration count are the two levers.

Contents

1 Overview

Phase 1 minimises the discrete Γ-convergence energy [1]

$$E_\varepsilon(u) = \varepsilon\, u^\top K u + \tfrac{1}{\varepsilon}\left(u^2(1-u)^2\right)^\top M \left(u^2(1-u)^2\right) + (\text{penalty}) \tag{1}$$

over nodal density functions, subject to a partition-of-unity constraint and an equal-area constraint, using projected gradient descent with an Armijo backtracking line search. The solver runs once per mesh refinement level, warm-starting each level from the previous one by nearest-neighbour interpolation.

Each major PGD iteration invokes a small set of operations, which become the canonical callback names used by the profiler:

Callback	**Operation**
`matrix_assembly`	FEM mass/stiffness assembly (once per level)
`projection`	orthogonal projection onto constraints (*per trial step*)
`energy`	energy evaluation $E_\varepsilon(u)$ (per trial step)
`gradient`	energy gradient $\nabla E_\varepsilon(u)$ (per iteration)
`backtrack`	the whole Armijo `while` loop (per iteration)
`init_interpolate`	inter-level nearest-neighbour interpolation
`h5_save`, `h5_flush`, `trigger_check`	I/O and plateau checks

This document reports only the *measured cost* of these operations.

2 Profiling Methodology

The class `RelaxationProfilingState` (`src/profiling.py`) accumulates wall-clock time and an invocation count per callback, *per refinement level*, plus three counters (`major_iterations`, `backtracks_per_iter_total`, `projection_inner_iters_total`). Results are written to `solution/timing_prof` when `--profile` is passed to `scripts/find_surface_partition.py`, or when `profile: true` is set in the experiment YAML (the mechanism used by the parameter sweep, which cannot inject CLI flags). The instrumentation is zero-overhead when disabled: every hook is a single `if profile is not None` guard, verified to perturb per-level wall time by $< 2\%$.

Nesting of callbacks (matters for accounting)

Two callbacks are recorded *inside* another and must not be double-counted: `energy` and `projection` are timed inside `backtrack` (every trial step evaluates both), and `triangle_areas` is timed inside `matrix_assembly`. The non-overlapping per-level sum (all callbacks except `energy`, `projection`, `triangle_areas`) closes to within 0.1–0.4% of the measured level wall time on every level of every run below — confirming the hooks capture essentially all wall time.

3 Reference Sweep

All numbers come from the 3×2 grid sweep in Table 1. The mesh schedule is fixed across all runs; only the penalty weight λ and the random seed vary.

Table 1: Reference sweep parameters.

Parameter	**Value**
Surface	torus ($R = 1$, $r = 0.6$)
Number of regions N	30
Refinement levels	3
Mesh vertices / level	2 760 → 12 688 → 29 808
Penalty weight λ	{1.9, 2.0, 2.1}
Seeds	{13131313, 84172851}
Projection inner cap	200 iters, tol 10^{-10}

4 Results

Table 2 gives the aggregate per-run breakdown. The `projection` share is remarkably stable at 86–88% regardless of λ, seed, or total runtime. The runtime itself, however, spans 4.3×, tracking `mean_backtracks` and the resulting projection-call count almost perfectly.

Table 2: Aggregate timing per run. "proj %" is projection share of total wall; `mBT` is mean backtrack trial steps per major iteration; `mPII` is mean projection inner iterations per call. Percentages sum with the unshown callbacks (energy, gradient, backtrack-net, overhead) to 100%.

λ	seed	total (s)	proj %	`mBT`	`mPII`	proj calls	major iters
1.9	13131313	3401	86.7	11.65	17.0	197 381	16 935
1.9	84172851	3008	86.5	11.14	17.4	172 103	15 444
2.0	13131313	1324	87.5	3.00	18.4	10 902	3 636
2.0	84172851	1014	86.1	2.91	17.4	11 032	3 787
2.1	13131313	3099	85.9	9.92	15.3	188 609	19 018
2.1	84172851	923	85.9	3.35	16.2	11 267	3 359

Figure 1 shows the per-level breakdown of a slow ("thrashing") run and Figure 2 a fast ("healthy") run. The contrast is the whole story: in the thrashing run the *coarsest* level 0 (2 760 vertices) consumes ∼ 75% of total time because the line search averages ∼ 12 trial steps per iteration (bottom-right panel) and grinds 15 590 iterations; in the healthy run the line search behaves (∼ 3 trials/iter) and cost falls on the finest level, as one would expect.

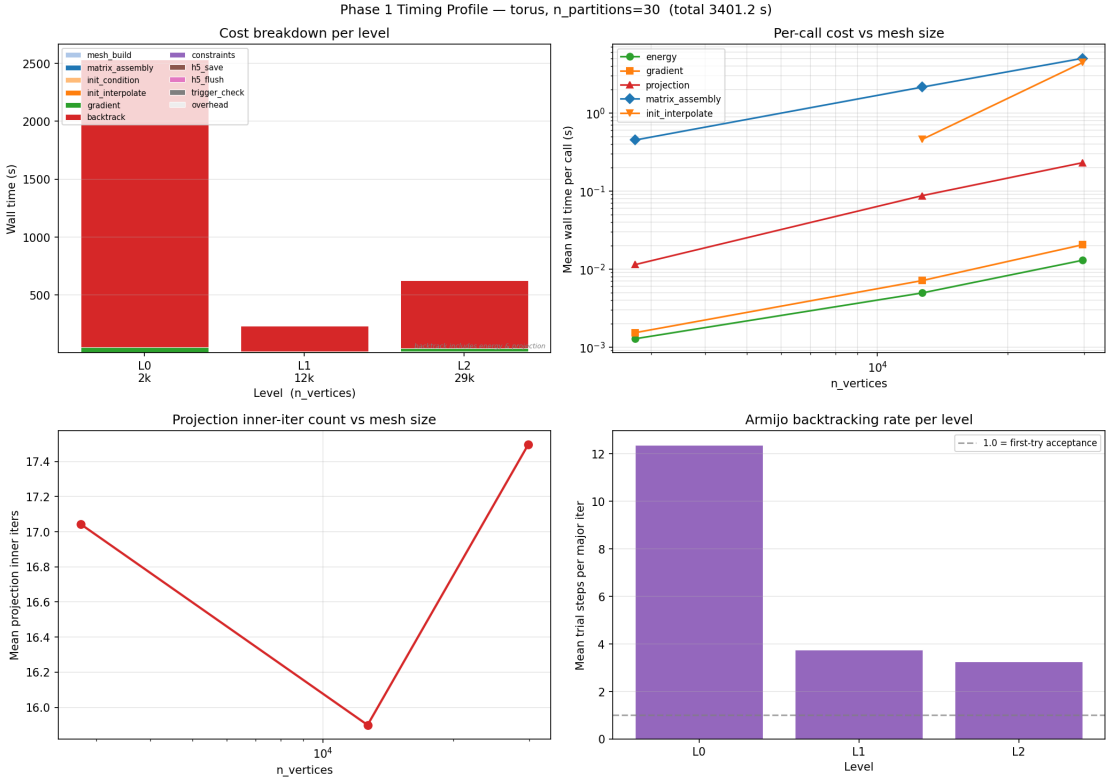

Figure 1: **Thrashing regime** ($\lambda = 1.9$, seed 13131313, total 3401 s). Top-left: level 0 dominates wall time. Bottom-right: the Armijo line search averages $\sim$12 trial steps per major iteration on level 0 (dashed line marks first-try acceptance). Each trial step triggers a full projection, so projection-call count — and thus runtime — explodes.

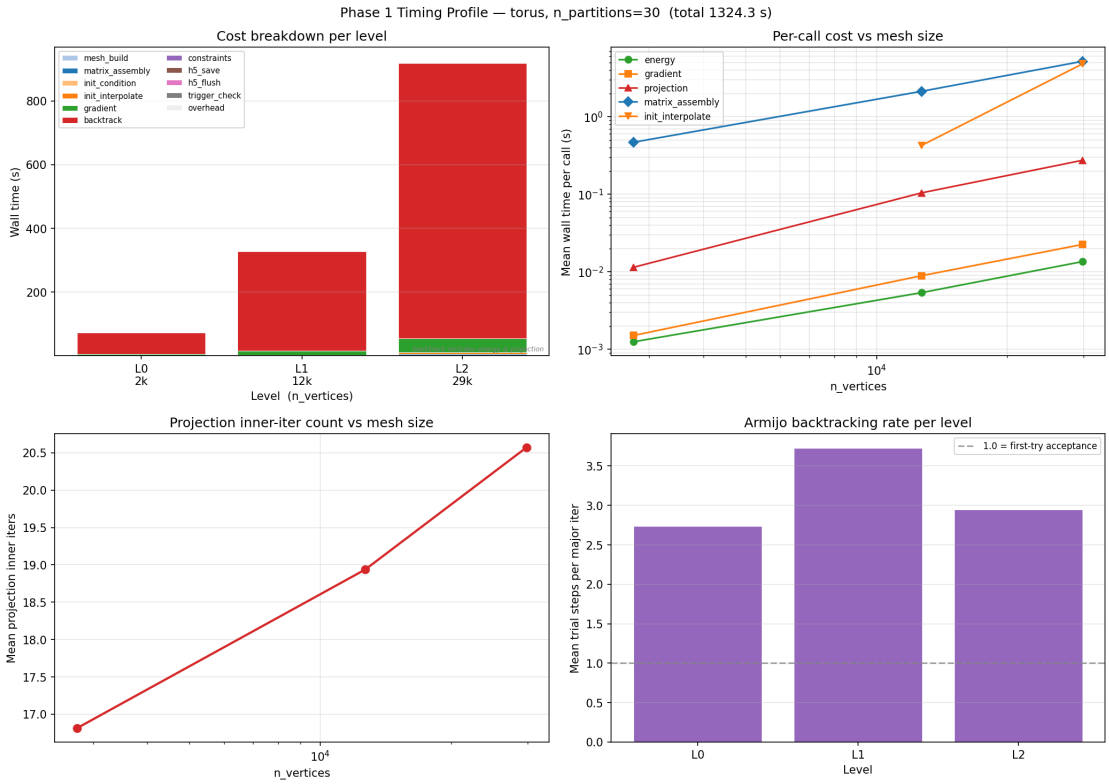

Figure 2: **Healthy regime** ($\lambda = 2.0$, seed 13131313, total 1324 s). Backtracking stays near 3 trials/iter at all levels, so cost falls on the finest level (top-left) as expected for a mesh-bound solver. Projection is still 87.5% of wall time — it is the bottleneck in *both* regimes.

5 Analysis: the Projection Bottleneck

Two facts together explain every run:

1. Projection is the dominant per-operation cost. Each `projection` call runs an iterative alternating projection (Dykstra-style) whose inner-iteration count (`mPII` in Table 2) is 15–20 and grows mildly with mesh size. Crucially, `mPII` is far below the cap of 200: projection is *not* hitting its iteration limit. The cost is therefore driven by the *number* of projection calls, not by any single call stalling.

2. The line search multiplies the projection count. The PGD step is

$$u^+ = \Pi_{\mathcal{C}}\big(u - s\,\nabla E_\varepsilon(u)\big), \tag{2}$$

and the projection $\Pi_{\mathcal{C}}$ is re-evaluated for *every* trial step size s during Armijo backtracking. Hence

$$\#\,\text{projection calls} \;\approx\; (\text{major iters}) \times \big(\texttt{mBT} + 1\big). \tag{3}$$

In the thrashing regime $\texttt{mBT} \approx 11$, so $\sim$90% of all projections are spent on *rejected* trial steps. This is why the $\lambda = 1.9$ run issues 197 381 projection calls versus 10 902 for $\lambda = 2.0$ — an 18× difference that maps directly onto the 2.6× runtime difference (the thrashing calls are cheaper because they sit on the coarse level 0).

Remark 5.1 (λ is not the control variable; the interaction is)**.** Total wall time is V-shaped in λ with a minimum at $\lambda = 2.0$ (Figure 3), but the effect is not monotone and is strongly seed-dependent: $\lambda = 2.1$ thrashes for seed 13131313 yet is the fastest run for seed 84172851.

Whether the line search thrashes is a property of the (λ, init) pair, not of λ alone. Only $\lambda = 2.0$ was well-behaved for both seeds.

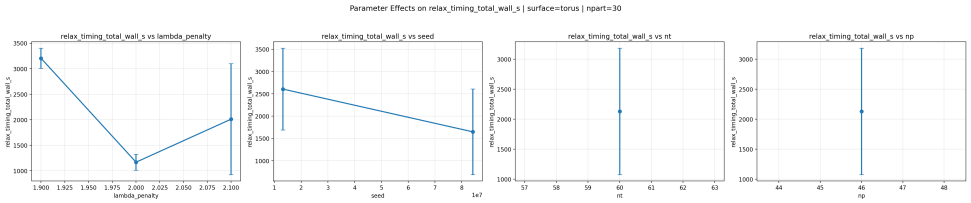

Figure 3: Total wall time versus λ (left panel; error bars span the two seeds). The minimum at $\lambda = 2.0$ and the large spread at $\lambda = 2.1$ reflect line-search thrashing, not algorithmic work that λ legitimately adds.

What is *not* a bottleneck. FEM `matrix_assembly` is 0.2–0.9% of wall time at these mesh sizes; energy, gradient, interpolation and I/O are each a few percent or less. The Python-loop assembly that document `02-phase2-timing-profile` anticipates as a future hot spot does not bite until 10^5–10^6 vertices, well beyond the 30k reached here.

6 Scaling Outlook: Increasing the Number of Regions

The motivating question is how Phase 1 cost behaves as the number of regions N grows. The projection operates on the $V \times N$ density matrix (V = vertices), so a single projection call costs $\mathcal{O}(\texttt{mPII} \cdot VN)$, and total Phase 1 cost is

$$T_{\text{phase 1}} \sim \underbrace{(\text{major iters})\,(\texttt{mBT}+1)}_{\#\text{ projections}} \times \underbrace{\texttt{mPII} \cdot VN}_{\text{cost per projection}} . \tag{4}$$

Every factor except V is tied to the projection, which gives a clear priority order for scaling up N:

1. **Eliminate re-projection on rejected steps.** Equation (3) shows the $(\texttt{mBT}+1)$ factor is pure waste in the thrashing regime. Caching/reusing the projection across backtrack trials, or a step-size rule that does not thrash (e.g. Barzilai–Borwein or an adaptive `step0`), removes the single largest source of variance and the bulk of the slow-run cost. Highest expected payoff.
2. **Warm-start the projection.** `mPII`= 15–20 inner iterations per call could be cut by warm-starting $\Pi_{\mathcal{C}}$ from the previous accepted iterate's multipliers, since consecutive iterates are close.
3. **Watch the N factor directly.** Cost per projection is linear in N, so doubling regions roughly doubles per-call cost *at fixed mesh*. Combined with the fact that more regions typically need finer meshes (larger V) to resolve interfaces, the VN product is the term that will dominate at large N — making items 1 and 2 more, not less, important as N grows.
4. **FEM assembly is a later concern.** Vectorising `matrix_assembly` matters only once V reaches 10^5–10^6; it is irrelevant at the current scale.

Summary table

Callback	**Current cost (this sweep)**	**Implication for scaling N**
`projection`	86–88% of wall, all regimes; cost $\propto$ `mPII` $\cdot\, VN$	Primary target: cache across backtracks; warm-start inner solve
`backtrack` (line search)	multiplies projection count by `mBT`+1 (3 to 12)	Non-thrashing step rule removes $\sim$ 90% of slow-run projections
`energy`, `gradient`	$\sim$5–9% and $\sim$3–5%	Minor; vectorise per-region matvec loop if needed
`matrix_assembly`	$< 1\%$ at $\leq$ 30k vertices	Only matters at 10^5–10^6 vertices

References

[1] Beniamin Bogosel and Édouard Oudet. Partitions of minimal length on manifolds. *Experimental Mathematics*, 26(4):496–508, 2017. YEAR CORRECTED 2026-08-21: this entry read year 2023 with no volume or pages, and its key was `bogosel2023partitions`. Crossref gives print publication 2 October 2017 (online 4 October 2016), Experimental Mathematics 26(4):496–508. This is the foundational reference for the Phase 1 relaxation and Phase 2 refinement methodology.